Provider implementation plan v2 — proposed assurance refinement

The approved functionality and layout from implementation plan v1 and scope v6 remain the baseline. This proposed refinement adds independent software review and a distinct hardening pass before laptop acceptance or a shareable claim. It responds to John's October 5 question about adversarial review and robustness. It does not approve or close any slice.

What the evidence establishes today

The original build landed and its required 55 tests passed. No independent adversarial code-review receipt has been established. A normally authenticated real-service retry timed out before refreshing native status. The follow-up repair has now landed, with 60 required tests and three browser tests passing. Its author traced repeated full-ledger scans under the provider lock and reports isolated rendering in 1.21 seconds cold and 0.10 seconds warm with over 15,000 real ledger files and simulated native contention. That repair remains unactivated: the unchanged live service took 17.1 seconds on its latest author check, and the existing laptop is unverified. Review must include both landed changes.

These tests provide useful evidence for cases they exercised. They do not prove robustness of untested failure paths, working enrollment, unsupported providers or an actual existing client.

Recommended execution and boundaries

Use Foreman's heavy build tier for the bounded review/fix/re-review work after the current repair finishes. Its deeper review is justified by credentials, separate account identities and a live failure in this area. The current selected families are ChatGPT for execution and Grok for independent review; re-resolve valid host preferences at launch, request latest supported models and highest supported effort, and record actual service. Two fresh, read-only reviewers must return substantive independent findings. Missing, same-family or failed reviewers cannot be described as cross-family assurance.

Crucible remains the option for reforging an unsettled design. The approved product design need not be reopened merely to examine the implementation. This proposal is a bounded Foreman contract, not a claim that Foreman has run. Its contract must be confirmed before launching it, and all long execution remains in a code job's isolated worktree. The steward updates authoritative campaign records only after repository ownership releases. Preserve shared staging and active conversations.

Frozen inputs are scope v6, this cumulative plan v2 and a new assurance execution log inside the assigned worktree. Run at most two waves, four fix iterations per wave and 120 minutes, then return the actual evidence or explicit halt. The native code-job runner owns landing; any Foreman commits stay inside that job's assigned worktree. The terminal gate is:

test-command: python -m pytest -q tests/test_provider_setup.py tests/test_provider_setup_browser.py tests/test_provider_setup_rigor.py tests/test_provider_setup_hardening.py

Needed because this software handles provider credentials and routes work between accounts; dropping independent review leaves the implementation judged only by its author. Needed because the real-service retry failed; dropping fault and concurrency tests leaves that class of failure unproved. Nothing

requested is cut; calendars/email, backups and collaboration retain their existing later positions. Backups are separate from software hardening.

Wave 1 — independent review with reproducible failures

Review the complete provider integration and the loading repair against cumulative scope v6, including settings, native adapters, secret storage, routes, usage attribution, Cave controls and engine routing. Reviewers challenge the actual changed code and tests, rather than endorse the author's summary. Findings carry file/line evidence, severity and a failing reproduction when possible. Record each response, actual reviewer family and any missing participation.

Add targeted tests where existing tests do not establish a required behavior. Required Test file `tests/test_provider_setup_rigor.py` with ids: cached-read-during-native-refresh; repeated-cached-page-load; separate-account-allowance-attribution; unsupported-is-not-usable; failure-keeps-existing-session-and-quality. Reproduce the observed loading symptom causally if possible; a timeout alone does not identify its cause. Use realistic temporary inputs, normal browser authentication and concurrent native-refresh simulations. Keep GET events soft-wakes and unrelated POST work starts blocked during display tests.

gate-command: python -m pytest -q tests/test_provider_setup.py tests/test_provider_setup_browser.py tests/test_provider_setup_rigor.py

The orchestrator records actual counts and runs the gate itself. Independent findings must be dispositioned with evidence; unresolved serious findings or missing required review keep the product unready. Do not weaken existing assertions or substitute a prose approval for a failing reproduction.

Wave 2 — harden the identified failure paths and re-review

Fix supported, in-scope findings and verify cancellation/timeouts, corrupt or changed settings, stale/unknown/exhausted allowance, account separation, secret redaction and credential-download rejection, and responsive cached reads while native work is slow. Use isolated temporary accounts and stores; real enrollment or credential replacement is not a test prerequisite. Exercise existing session identity, quality settings and drafts through those failures.

Required Test file `tests/test_provider_setup_hardening.py` with ids: native-timeout-retains-cached-state; cancelled-owned-process-cleanup; invalid-settings-do-not-overwrite; credentials-not-in-responses-or-artifacts; account-selection-preserves-running-conversation; cached-state-does-not-wait-for-refresh. Write meaningful failure tests for the actual mechanism, not assertions that mirror a chosen implementation. Add only the production changes needed by demonstrated findings.

gate-command: python -m pytest -q tests/test_provider_setup.py tests/test_provider_setup_browser.py tests/test_provider_setup_rigor.py tests/test_provider_setup_hardening.py

Re-review the fixes and affected seams with the independent family. Preserve finding IDs, reproduction commands and test results. Zero unresolved blocker/major findings and complete required reviewer participation are necessary for technical readiness; a deterministic pass does not invent reviewer convergence.

Existing roadmap and human acceptance

Keep `gwl-provider-status` open for the present build/repair. Add `gwl-software-review` (part=rigor) and `gwl-provider-harden` (part=harden) immediately after it, with the two commands above. Keep `gwl-review` as the existing explore/refine surface (part=slice), and keep `gwl-laptop-provider-test` (part=rigor) for actual existing-client proof. The original provider, Cave, organization, optional accounts and backup steps retain their scope.

After technical review, safely activate only with John's restart instruction or an already established idle-restart authorization. Verify repeated normally authenticated real-service reads and the existing laptop client's gear, saved accounts, allowance/error states, model controls and retained draft. A fresh browser or worktree screenshot cannot substitute for that device. Give John the working surface for feedback and fold refinements into this same effort. Sharing still needs Distribution's independent second-device connection trial. No automatic publication or collaboration scope is added.

Technical readiness is bounded evidence, not a guarantee of defect-free software. The product is unaccepted until John's feedback and explicit close; Foreman's technical completion must not close the cockpit slice. Implementation plan v1 remains the approved plan while this v2 proposal awaits contract confirmation.

Approved functional baseline — implementation plan v1

John accepted Setup mockup Refinement 2 as the starting layout and explicitly authorized its build and integration in turn `d7dd98a2-97a2-42f9-9273-c71b3e49cbf9` (October 4, 2026, Mountain time). The goal remains one personal mother ship holding provider credentials and running AI work for connected devices. This plan extends the existing roadmap, rather than starting another effort.

What this build delivers

Keep the approved Setup page, gear and Cave controls, and connect them to real host-side provider settings. Existing native sign-ins should work without keeping terminal windows open. New sign-ins use the provider's normal browser/device flow; separately named subscriptions retain separate identity and reports. Explicit API connections retain their distinct billing mode.

Existing roadmap step	Part	Implementation and proof
gwl-access	integrate	Read existing sign-in status safely, add native login and supported account isolation; preserve the default account and active sessions.
gwl-provider-status	slice	Show real account-qualified allowance/reset reports where supported, separate observed tokens and API spend, and identify unknown, unsupported or stale reports. This remains the current ongoing slice.
gwl-cave-settings	integrate	Connect the upper-left gear and bottom-left provider/model controls to the same Setup settings; retain the separate capacity display and Cave dialogue state.
gwl-laptop-provider-test	rigor	Check routes, credentials isolation and real UI in the worktree, then verify the existing running laptop client after safe activation.
gwl-review	rigor	Let John use and refine the integrated product; preserve the accepted layout and earlier requirements. No slice closes without his explicit instruction.

The code job owns implementation in its own worktree, using Anchor's latest selected coding model at maximum effort. Its gate is `python -m pytest -q tests/test_provider_setup.py`, with focused affected regressions and normally authenticated browser checks. A failed gate prevents automatic landing. Do not restart active sessions to activate code; inspect the actual change and use a safe boundary.

Capability and scope limits

Codex documents account, rate-limit and usage reads; verify the installed version before promising a working meter. Claude's existing limit events need account qualification. Grok's native usage support must be verified. Unsupported fields stay explicit, never estimated as available allowance. Account authentication and quota availability are separate facts.

Keep the settled warning policy: full strength remains until the user explicitly changes it. Preserve existing projects, conversation identities, goals and pairing. No new installations, credential replacement, file consolidation or live provider work starts merely from opening settings or refreshing status. Native

browser/device login is distinct from creating an independent OAuth client.

Nothing requested was cut. Calendars/email, backups and cross-mother-ship collaboration remain in their existing later places. Needed because live activation must preserve ongoing work; dropping the activation check risks interrupting active conversations.

Sources: Codex app-server, Codex authentication, Claude authentication, and the complete original effort conversation and scope v5. Scope v6 records this turn's accepted layout and implementation authorization.